

CONTENTS

- Overview
- 1 Refactoring: change the inside, preserve the outside
- 2 Why bother, and what happens if you do not
- 3 Inline versus agentic refactoring
- 4 The risk: a probabilistic system editing hundreds of files
- 5 The loop, part one: plan, read, search, report
- 6 The loop, part two: approve, snapshot, patch
- 7 Verify, roll back, and the CI/CD fit
- 8 Deterministic patching with syntax trees
- 9 Learning from accept, reject, pass and fail
- Glossary
- Check yourself
- Where to go next

AI Code Refactoring: Letting a Probabilistic System Edit Production Code Safely

Understand the loop, the verification steps and the deterministic tooling that make autonomous refactoring trustworthy.

For engineers evaluating or building autonomous refactoring into their workflow · 26 July 2026

Every section, key point, term and question from the full lesson is here. The extended explanations and the additional examples are not.

[Watch the source video](#)

[Read the full lesson](#)

[Read the highlights](#)

BEFORE YOU START

- Working experience with a real codebase and a test suite
- Familiarity with version control and CI
- Comfort reading Python

Overview

A large language model is a probabilistic guessing machine, and these machines are now being pointed at production code. The obvious question is whether that is safe.

The answer turns out not to be about the model at all. Refactoring means changing a program's internal structure while leaving its external behaviour identical, and 'identical' is a checkable property — you have tests, a build, a type checker and a diff. Agentic refactoring is safe to the extent that a guess has to survive those checks before it reaches the codebase, and unsafe to the extent that it does not.

So the interesting content is the loop: plan, read, search, report, approve, snapshot, patch, verify, roll back on failure and go round again. Two details sharpen it further. The patch step can skip probabilistic generation entirely by operating on a syntax tree, making common transformations deterministic. And every accept, reject, pass and fail is a training signal, so the suggestions improve over time.

KEY TAKEAWAYS

- ✓ Refactoring changes internal structure while leaving external behaviour identical — the second half is what makes it verifiable.
- ✓ The three canonical motives are readability, removing duplication and lowering complexity; skipping them compounds into technical debt.
- ✓ Inline refactoring works alongside a developer on small local edits; agentic refactoring is handed a goal and works the whole codebase autonomously.
- ✓ The danger is a probabilistic system deleting things that look unused but exist for a reason nobody wrote down.
- ✓ The guard rail is a loop: plan, read, search, report, approve, snapshot, patch, verify — with rollback on failure.
- ✓ Human approval sits between the report and any edit, so guesses never reach the codebase unreviewed.
- ✓ Because verification is just tests and builds, the whole loop slots into CI/CD like another developer.
- ✓ Patching via a syntax tree makes transformations like rename deterministic rather than probabilistic.

01 Refactoring: change the inside, preserve the outside

0:00

Take a program's internal structure and change it, while to the outside world the thing still looks exactly the same.

KEY POINTS

- Refactoring changes internal structure and preserves external behaviour.
- Preservation is what makes correctness checkable rather than a judgement call.
- External behaviour includes side effects, errors, types and timing — not just return values.
- Verifiability depends on being able to detect a behaviour change, which usually means tests.
- Refactoring untested code is rewriting, whoever performs it.

The invariant, stated as a test

This is the shape of a characterisation test: it does not assert that the behaviour is correct, only that it has not changed. For code you are about to restructure, that is exactly the right assertion, and it can be generated from recorded production inputs.

```
@pytest.mark.parametrize("case", RECORDED_INPUTS)
def test_behaviour_unchanged(case, snapshot):
    result = process(case.input)

    assert result == snapshot(case.id)          # same return value
    assert case.side_effects == captured_writes() # same writes
    assert case.raised == captured_exceptions()  # same errors

# it does not claim the behaviour is right. it claims it is the same,
# which is the only property a refactoring is allowed to preserve.
```

02 Why bother, and what happens if you do not

0:43

Readability, removing duplication, lowering complexity. Skip them long enough and it compounds into technical debt.

KEY POINTS

- Readability: names that carry meaning reduce the cost of every future read.
- Duplication: one copied bug becomes several, and nobody knows how many.
- Complexity: a function doing six jobs is hard to test and unsafe to change.
- Technical debt is the compounding cost of skipping this work.
- The cost is the interest — paid by every future change, invisible in any single sprint.

The three canonical moves

Each preserves behaviour exactly and each is mechanical enough to automate reliably — which is precisely why refactoring was the first place agentic coding tools became genuinely useful.

```
# 1. rename for readability
- temp2 = fetch(o.cid).st
+ customer_state = fetch(order.customer_id).state

# 2. remove duplication – the same 20 lines existed in 4 files
+ def normalise_address(raw): ...
- <the same block, inlined, four times, one of them subtly different>

# 3. lower complexity – one 300-line function doing six jobs
+ def validate(order): ...
+ def price(order): ...
+ def apply_discounts(order): ...
+ def charge(order): ...
- def process_order(order): # 300 lines, six responsibilities
```

03 Inline versus agentic refactoring

2:50

One works next to you on small local edits. The other is handed a goal and works the whole codebase on its own.

KEY POINTS

- Refactoring is largely pattern recognition, which is what these models do well.
- Inline refactoring: small local edits in the editor, alongside the developer.
- Agentic refactoring: given a goal, works the whole codebase autonomously.
- Goals range from precise ('upgrade this library') to vague ('clean up this module').
- The real difference is the review surface: continuous and implicit, versus deferred and deliberate.

Two tools, two working modes

The last row is the one that determines how much machinery you need around the tool. A rejected inline suggestion costs a keystroke; a bad agentic change set costs a review of hundreds of files.

	inline	agentic
input	cursor position	a goal, in English
scope	one expression, one file	the whole repository
autonomy	suggests, you accept	plans and executes
duration	instant	minutes to hours
context held	the current file	project and dependencies
review	continuous, implicit	deferred, deliberate
cost of a bad edit	one keystroke	a change set to audit

04 The risk: a probabilistic system editing hundreds of files

4:55

It predicts likely-looking code, so it may tidy away something that looked unused but was there for a reason.

KEY POINTS

- The model predicts likely-looking code; it does not know why something exists.
- The leap-year example is locally correct reasoning reaching a wrong conclusion.
- Watch for code reached by reflection, schedulers, queues, flags, shims and rare error paths.
- The risk is not new — the volume and speed are.
- The mitigation is verification before commit, not a more careful model.

Correct reasoning, wrong answer

Reconstruct the agent's chain and every step is defensible. The missing information was never in the repository, which is why no amount of reading the code would have prevented it — and why the loop needs steps that are not reading the code.

```
def adjust_for_leap_year(date, term_days):
    """Contract terms crossing 29 Feb need a day added."""
    ...

# the agent's reasoning:
# 1. grep for adjust_for_leap_year → one definition, no calls
# 2. no tests reference it
# 3. no imports elsewhere in the repo
# 4. removing unused code is a valid refactoring
# → remove it

# what it could not see:
# called via getattr(rules, f"adjust_for_{rule_name}") in a config-driven
# dispatcher, where rule_name comes from a YAML file
```

05 The loop, part one: plan, read, search, report

5:37

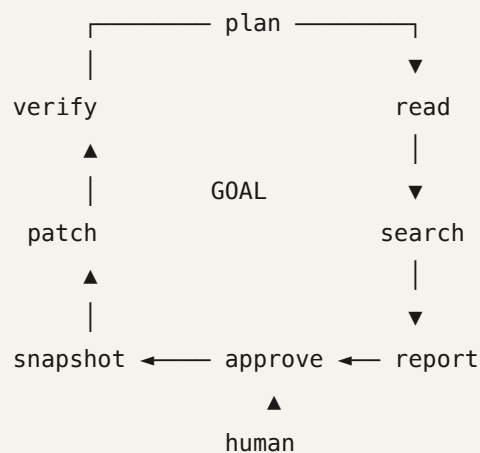
Agentic refactoring manages risk by working as a cycle. The first four stages build context and produce findings rather than edits.

KEY POINTS

- The loop runs plan → read → search → report, with the goal at the centre.
- Plan produces an ordered list of changes and is revised as the agent learns.
- Read maps structure: functions, classes and their relationships.
- Search covers the rest of the project and its dependencies to find all references.
- Report ranks findings by priority, each with explicit reasoning.
- None of these four stages modifies anything — the output is a document.

The loop

Read it as a cycle rather than a pipeline. A failure at verify does not abort the job — it returns to plan with new information, which is what makes the agent able to recover rather than merely stop.



```
# read / search / report change nothing.
# nothing is written until after approve.
```

06 The loop, part two: approve, snapshot, patch

7:44

The human re-enters here, choosing which findings to fix. Then a snapshot is taken so everything can be undone, and only then does code change.

KEY POINTS

- Human approval sits after analysis and before any modification.
- The human reviews a ranked list with reasoning, not a large diff.
- A snapshot is taken automatically so every change can be undone.
- Patching carries the context gathered during read and search, unlike find-and-replace.
- Keep change sets small — approval is worthless if the diff is too large to read.

Approval as a selection, not a yes/no

Selecting a subset is what makes the review real. A single accept-everything button trains the human to click it, which removes the guard rail while leaving it visible in the diagram.

Findings for goal: "clean up billing/"

[x]	HIGH	charge.py:88	bare except swallows gateway timeouts
[x]	HIGH	invoice.py:112	duplicated day-count logic (4 sites)
[]	MED	refund.py:203	extract 180-line function ← not now
[x]	LOW	monthly.py:12	unused import
[]	LOW	legacy.py:41	remove apparently unused helper ← verify first

→ 3 of 5 approved. the agent patches only those.

07 Verify, roll back, and the CI/CD fit

8:27

The agent writes and runs tests and rebuilds the project. If a test breaks it rolls back and goes round again — which is why the whole loop fits into a pipeline.

KEY POINTS

- Verify runs the tests and rebuilds the project, then shows a diff against the snapshot.
- A failing test triggers rollback and another pass through the loop.
- Guesses face two independent gates: human approval and automated verification.
- Because verification is tests and builds, the loop fits into CI/CD like another developer.
- Verification is only as strong as the test suite — add characterisation tests first where coverage is thin.
- Type checks, linters, builds and contract tests are additional cheap gates.

Verification as a stack of independent gates

Each gate catches a different class of mistake and each is cheap. On codebases where coverage is weak, the gates other than tests are what keep the floor off the ground.

```
def verify(snapshot):
    checks = [
        ("build",      lambda: run("make build")),      # imports, syntax
        ("types",      lambda: run("mypy .")),          # signature drift
        ("lint",       lambda: run("ruff check .")),     # obvious removals
        ("unit",       lambda: run("pytest tests/unit")), # behaviour
        ("contract",   lambda: run("pytest tests/contract")), # external API
    ]

    for name, check in checks:
        if not check():
            git.reset_hard(snapshot.base)                # roll back
            return Failure(stage=name)                   # and loop again

    return Success(diff=git.diff(snapshot.base))
```

08 Deterministic patching with syntax trees

9:53

The patch step can skip probabilistic generation entirely by parsing the code into a tree and operating on its structure.

KEY POINTS

- Parsing to a syntax tree makes transformations structural rather than textual.
- A rename becomes: find the symbol, update every reference — correct by construction.
- Text replacement cannot distinguish variables, fields, substrings and comments; a tree can.
- A type-aware tree distinguishes identically named symbols on different types.
- Lossless trees preserve formatting and comments, keeping diffs small and reviewable.
- Let the model decide what to change; let the tree perform the change.

Four things text replacement cannot tell apart

Every line contains the string. Only one should change. This is why mechanical refactorings performed by text substitution produce bugs that survive review — the wrong edits look exactly like the right ones.

```
customer_state = order.state      # the variable – rename this
shipping.state = "CA"            # a different object's field
real_estate_value = 100          # a substring of another identifier
# the state machine handles this  # a comment
settings["state"] = active        # a string key

# sed 's/state/status/g' changes all five.
# a tree operation changes one.
```

09 Learning from accept, reject, pass and fail

10:35

Every approval, rejection, passing test and failing test is a training signal, so the suggestions improve over time.

KEY POINTS

- Accept/reject decisions and test pass/fail results are both training signals.
- The loop produces labels as a by-product of ordinary work, with no annotation effort.
- Test outcomes are objective, automatic and high-volume — closer to a grader than a preference.
- The signal inherits your reviewers' blind spots, including a habit of approving in bulk.
- It also inherits your coverage: undetectable breakage is labelled as success.
- Verification and human review are what make the capability usable, and what make it improve.

Labels generated by using the system

Four rows, four labelled examples, and nobody was asked to annotate anything. The last row is the most informative kind: a change that looked right, was approved by a human, and was caught by the suite.

finding	approved?	patched?	tests	
bare except in charge.py	yes	yes	pass	+
duplicated day-count logic	yes	yes	pass	+
extract 180-line function	no	—	—	—
remove "unused" helper	yes	yes	FAIL	--

the last row is worth the most: plausible, human-approved,
and objectively wrong. that is a hard negative you cannot buy.

Glossary

Refactoring

Changing a program's internal structure while leaving its external behaviour identical.

External behaviour

Everything observable from outside: return values, types, side effects, exceptions, emitted logs and timing. What a refactoring must preserve.

Technical debt

The compounding cost of deferred cleanup, paid as interest by every future change that has to route around the mess.

Characterisation test

A test asserting that behaviour has not changed rather than that it is correct. The right instrument before refactoring untested code.

Inline refactoring

Small, local suggestions made in the editor alongside the developer, reviewed continuously as they appear.

Agentic refactoring

An autonomous agent given a goal that plans and executes changes across the whole codebase, reviewed after the fact.

The refactoring loop

Plan, read, search, report, approve, snapshot, patch, verify — with rollback on failure and another pass round.

Snapshot

The recorded starting state, in practice a branch or commit, taken automatically so any change can be undone.

Abstract syntax tree (AST)

A structural representation of parsed code. Discards formatting and comments, so regenerating source reformats the file.

Lossless syntax tree

A richer tree that preserves formatting, comments and whitespace exactly, so a transformation produces a minimal diff.

Deterministic patching

Applying a change as a tree operation rather than generating text, making transformations like rename correct by construction.

Dead code

Code with no reachable callers. Hard to establish statically, because reflection, schedulers, flags and config can reach it invisibly.

Training signal

An outcome usable as a label. Here: whether a finding was approved, and whether the resulting patch passed the tests.

Check yourself

Answer first, then open each one.

Why does refactoring's definition — internal change, external behaviour preserved — make it a good first target for autonomous AI tools?

Because the success criterion is mechanically checkable. You are not asking whether the new code is good, only whether it behaves identically, and tests, types and builds can answer that automatically. A task with an automatic correctness check is one where a probabilistic system can be allowed to guess, because the guess gets verified before it counts.

The agent removed a function with no callers, no tests and no imports, and it was needed. Where exactly did the reasoning fail?

It did not fail. Every step was locally correct; the justification for keeping the function existed outside the repository, in a config-driven dispatcher or an incident three years ago. That is why the loop needs stages beyond reading code — searching dependencies and dynamic references, plus a human who might recognise the name.

Why does human approval sit between the report and the patch rather than after the patch?

Because it puts the review before the irreversible step, and it makes the review a ranked list with reasoning rather than a diff across hundreds of files. The expensive autonomous analysis happens unsupervised, which is fine because it changes nothing; the human's attention is spent only on the decision that has consequences.

Your module has 21% coverage. Why does a passing verify stage prove almost nothing, and what should happen first?

Verification detects behaviour changes only where tests observe behaviour, so most of the module's behaviour could change without any test noticing. The correct sequence is two separate reviewed changes: first the agent writes characterisation tests capturing current behaviour, which a human reviews; then it refactors against them. PR 1 is easy to review precisely because nothing behavioural changed in it.

Why is renaming a variable via a lossless syntax tree better than a careful find-and-replace, even when the replacement is scoped to one file?

Because text has no idea what any occurrence is. The same string appears as your variable, as a different object's field, inside a longer identifier, in a comment and as a dictionary key — and the wrong edits look exactly like the right ones in review. A tree operates on identifier nodes, so it changes only the symbol, and a lossless tree keeps the rest of the file byte-identical so the diff shows nothing else.

The loop generates training signals from approvals and test results. What does that signal inherit that you should audit?

Your review culture and your coverage. If reviewers approve in bulk, the model learns that most proposals are acceptable and proposes more of the same. If coverage is thin, patches that break untested behaviour are labelled as successes, teaching the model that undetectable breakage is fine. Both failure modes reinforce themselves.

Where to go next

- Take a module you are afraid to change, have an agent write characterisation tests against its current behaviour, and read them — the tests alone usually reveal behaviour nobody knew was there.
- Write a libcst transformer for a mechanical migration you have been putting off, and compare its diff against what a text-based script produces on the same repository.
- Run an agentic refactoring with a deliberately vague goal on a scratch branch and read the report without approving anything; the findings tell you as much about the codebase as about the tool.
- Audit your repository against the dead-code checklist and count how many entry points are invisible to static analysis — the number determines how much you can trust automated deletions.
- Wire the loop into CI on a schedule with a hard cap on findings per run, and measure how many of its pull requests get merged unchanged over a month.
- Compare the diff size of an AST-based transformation against a lossless-tree one on the same change, and notice how much reviewability depends on that difference.
- Read Fowler's **Refactoring** for the catalogue of named transformations — agentic tools implement these, and knowing the names makes their reports far easier to evaluate.

Lesson generated from a YouTube transcript with YouTube Lesson Builder.

Explanations are AI-generated. Check anything you intend to rely on.